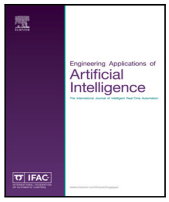
Engineering Applications of Artificial Intelligence 120 (2023) 105891

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai



Multi-UAV trajectory optimizer: A sustainable system for wireless data harvesting with deep reinforcement learning☆

Mincheol Seong [a], Ohyun Jo [b], Kyungseop Shin [c,*]

[a] The Republic of Korea Army, Daejeon, The Republic of Korea
[b] Department of Computer Science, Chungbuk National University, Cheongju, The Republic of Korea
[c] Department of Computer Science, Sangmyung University, Seoul, The Republic of Korea

ARTICLE INFO

Keywords:
Wireless sensor network
UAVs
Data harvesting
MARL
Transfer learning
Scenario parameters

ABSTRACT

A wireless sensor network assisted by multiple autonomous unmanned aerial vehicles (UAVs) is a promising solution for harvesting data and monitoring the circumstance in various applications. However, the complicated path planning problem of each UAV is still problematic. In this paper, we propose an optimal operation strategy based on multi-agent reinforcement learning (MARL) to tackle these hurdles. Various parameters such as the number of deployed UAVs, charging start capacity, and charging complete capacity define a multi-UAV system. This approach is applicable without a time-consuming and costly policy control. We also describe how to balance multiple objectives, such as data harvesting, charging, and collision avoidance, using transfer learning. Finally, learning a policy control that generalizes multiple scenario parameters allows us to analyze the performance of individual parameters in a specific scenario, which helps find the macro-level optimal parameter within a particular scenario. Videos are available at https://github.com/mincheolseong/UAV-Trajectory-Optimizer.

1. Introduction

There are many fields where data harvesting applications are needed; preventing large-scale disasters, unmanned farming systems, watching in the military system, monitoring natural environment changes, et al.. For example, it is essential to continuously monitor changes in the sea environment, such as climate change or contaminant detection. Many studies have been conducted for dynamic information monitoring systems in the sea environment through wireless sensor nodes (Alkandari et al., 2012; Bottarelli et al., 2019; Liu et al., 2010). It is still challenging for sensor nodes to collect the data continuously produced in a sea environment due to the limited transmission energy and the long distance from a receiver located on land. Autonomous unmanned aerial vehicles (UAVs) as data harvesters can be one of the promising solutions to resolve these problems. They have many advantages over satellites and ground base stations (Kishk et al., 2020). A quick and straightforward deployment would cover a wide range of areas and reduce the cost of the infrastructure installed on the land.

This paper focuses on building a UAV-aided data collection system based on a deep reinforcement learning mechanism to maximize its sustainability. A team of UAVs consists of a variable number of homogeneous drones with the same task of collecting data. While carrying out these tasks, the UAVs need to return to the charging station when the battery gets low. This issue imposes complex constraints on the design of the trajectory of the UAV for optimal data collection with limited battery capacity. Moreover, avoiding collisions between the UAVs should be taken into consideration to optimize the efficiency of the network when the number of UAVs increases.

In this regard, deep reinforcement learning (DRL) might provide an opportunity to alleviate the challenges to our goals. First, it can be applied to complex problems with many constraints through an intuitive combination of reward functions. In addition, bearing in mind that the UAV control problems in the communication scenario have been proven to be NP-hard in many cases (Zeng et al., 2019; Zhang et al., 2019), the DRL is computationally efficient for approaching such kind of NP-hard problem as shown in Shen et al. (2019).

1.1. Related work

Multi-UAVs are actively being studied for their flexible and efficient operations in various fields, including communications and networks (Pérez-Carabaza et al., 2019; Skorobogatov et al., 2020). In Fu et al. (2021), the authors suggested a method for a UAV to collect data through wireless charging. They adopted a single agent approach using Q-learning. However, it is impossible to consider all the complex

☆ This work was supported by the National Research Foundation of Korea [grant number 2021R1F1A1064059].
* Corresponding author.
E-mail addresses: smc5741@gmail.com (M. Seong), ohyunjo@chungbuk.ac.kr (O. Jo), ksshin@smu.ac.kr (K. Shin).

https://doi.org/10.1016/j.engappai.2023.105891
Received 6 September 2022; Received in revised form 18 January 2023; Accepted 19 January 2023
Available online 27 January 2023
0952-1976/

constraints and requirements in reality with the single agent approach. There have been studies on approaches using DRL in UAV-aided networks to describe the complex environment in reality (Lahmeri et al., 2021; Ullah et al., 2020). They considered convolutional neural networks and partial observability to define the environment as a partially observable Markov decision process (POMDP) for multi-UAV path planning. In a vast scenario parameter space in which environmental parameters are changed, there is no need for a separate relearning to be introduced in Bayerlein et al. (2021). It is possible to measure the performance of various scenario parameters. By the way, multiple objectives including data harvesting, charging, and collision avoidance have not been considered, which makes multi-UAV unsustainable. In addition, the agents' action set is limited to cardinal directions, making it challenging to clarify the agent's practical behaviors.

Multiple objectives within the DRL approach are studied in Punma, Unpublished results (0000), where Deep Q-Network (DQN) and transfer learning are leveraged in a multi-agent setting. They tried to achieve multiple objectives for autonomous vehicles to fulfill customer demands for charging in 7×7 or 10×10 grid environments. However, to optimize the charging system, it needs to consider more realistic environments with large grid numbers or detailed charging conditions.

We focus on constructing a sustainable UAV-aided network system by training UAVs with multiple objectives, including data collection, navigation constraints, and charging in a dynamic sea environment. To the best of the authors' knowledge, this study is the first to address multi-UAV path planning based on the collective combination of a multi-agent DRL and a transfer learning in which the learned control policies can achieve multiple objectives.

1.2. Organization

The rest of this paper is organized as follows. In Section 2, we describe the system model to present the UAV, network environment, and channel. All the system parameters are reflected in the POMDP to consider our practical DRL application scenarios in Section 3. The multi-agent DRL learning approach and transfer learning to optimize the system are introduced in Section 4. Simulation results are presented and analyzed in Section 5. Our conclusions and prospects for future work are given in Section 6.

2. System model

2.1. Environmental model

In this section, a system model is presented to make the RL approach applicable to the real world's dynamics. The architecture and the parameters are designed for applying the RL method to our environmental model.

The first step required to apply the RL model is to create an environmental model that reflects the behavior of the agents which is likely to occur in real environments. This environmental model contains places where UAVs can take off and charge and sensing points where sensor nodes are located. An example of a grid world is illustrated in Fig. 1, and a single UAV trajectory is marked as described in the attached legend in Table 1.

2.2. UAV model

The UAVs are placed and move within the grid world with a size of $M \times M$. The state of UAV i at time t, $\boldsymbol{s}_i(t)$, includes the following three elements:

(1) position $\boldsymbol{p}_i(t) = [x_i(t), y_i(t), z_i(t)]^T \in \mathbb{R}^3$, where the altitude $z_i(t) \in \{h\}$ is a constant altitude h;
(2) operational status $\boldsymbol{\phi}_i(t) \in \{0, 1\}$, either inactive or active;
(3) battery energy level $\boldsymbol{b}_i(t) \in \mathbb{N}$.

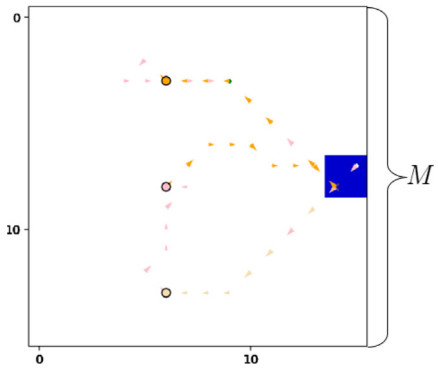


Fig. 1. Example of a single UAV system of size 16×16 collecting data from sensor nodes with a blue-colored start and charging zone **L**. Colored circles refer to sensor nodes, and different colors for each node make it easy to figure out which sensor node the UAV is communicating with.

Table 1
Legend for scenario plots.

	Symbol	Description
DQN	■	Start and charging zone
Input	●	Sensor node
Visualization	◇	Starting positions during an episode
	◆	Termination of an episode
	→	UAV movement while comm. with red node
	×	Hovering while comm. with red device
	★	Hovering while charging

Total operation time is set to $\boldsymbol{T}$ time steps for all UAVs. A mission time step with length δ_t is divided into m communication time steps. We use a simple version of charging, considering a partial charging principle and serving time from Zhang et al. (2018). The partial charging principle is to charge only the needed amount without a full charge when the energy level is low. In realistic environments, the battery charging process requires service time, expressed as the serving time. Now, we employ $\boldsymbol{b}_i^n(t)$ to represent a remaining energy ratio as $\boldsymbol{b}_i^n(t) = \boldsymbol{b}_i(t)/\boldsymbol{b}_i(0)$ (%). When the UAV starts charging or completes charging, $\boldsymbol{b}_i^n(t)$ is defined as charging start capacity or charging complete capacity, respectively, and is described as follows:

(1) charging start capacity (%) $\mathbf{C}_S$;
(2) charging complete capacity (%) $\mathbf{C}_C$.

For instance, $\mathbf{C}_S = 20$ means that the UAV can charge its energy from 20% of the full battery capacity, and $\mathbf{C}_C = 70$ means that the UAV charges up to 70% of the full battery capacity. Here, we define the relation between $\mathbf{C}_S$ and $\mathbf{C}_C$ as

$$10 \leq \mathbf{C}_S < \mathbf{C}_C \leq 100, \tag{1}$$

where $\mathbf{C}_C$ is always greater than $\mathbf{C}_S$, and UAVs cannot charge their energy beyond 100% of the entire battery capacity.

Each UAV is authorized to choose their action from action spaces, $\mathbf{A} = \{North, East, South, West, North-East, South-East, North-East, South-West, Hover\}$. While the UAV is charging, the required action of the UAV is $Hover$. In order to force the action to $Hover$ regardless of which actions a UAV selects in the action space $\mathbf{A}$, each UAV's physical action $\mathbf{a}_i(t) \in \widetilde{\mathbf{A}}(\boldsymbol{s}_i(t))$ can be limited to (2) if a charging condition $\mathbf{C}_i(t)$ is satisfied,

$$\widetilde{\mathbf{A}}(\boldsymbol{s}_i(t)) = \begin{cases} Hover, & \mathbf{C}_i(t) = 1 \wedge \ \boldsymbol{b}_i^n(t) \leq \mathbf{C}_C \\ \mathbf{A}, & otherwise, \end{cases} \tag{2}$$

which means that once a UAV starts charging, it has to keep charging until $\boldsymbol{b}_i^n(t) = \mathbf{C}_C$.

The charging determination $\iota_i(t)$ is required to start charging when $\boldsymbol{b}_i^n(t)$ is lower than $\mathbf{C}_S$ and to charge the battery until it reaches $\mathbf{C}_C$.

$$\iota_i(t) = \begin{cases} 1, & \boldsymbol{b}_i^n(t^p) \leq \mathbf{C}_S \\ 0, & \boldsymbol{b}_i^n(t) > \mathbf{C}_C, \end{cases} \tag{3}$$

where t^p is a fixed prior time slot before t. Note that the default value of $\iota_i(t)$ at the beginning of an episode is 0.

The charging condition $\mathbf{C}_i(t)$ is defined as follows in which the ith UAV starts to charge its energy only when the following conditions are satisfied.

$$\mathbf{C}_i(t) = \begin{cases} 1, & \boldsymbol{p}_i(t) \in \mathbf{L} \\ & \wedge\ \iota_i(t) = 1 \\ & \wedge\ \boldsymbol{a}_i(t) = Hover \\ 0, & otherwise. \end{cases} \tag{4}$$

The maximum displacement with which a UAV can move in one mission time slot is the grid cell size c. The speed of a UAV $\mathbf{v}_i(t)$ is constant within a time slot by dividing enough communication time slots. $\mathbf{v}_i(t)$ is upper bounded to moving at a horizontal velocity $V = c/\delta_t$ or hovering, i.e, $\mathbf{v}_i(t) \in \{0, V\}$ for all $t \in [0, T]$.

The position of the UAV is updated by

$$\boldsymbol{p}_i(t+1) = \begin{cases} \boldsymbol{p}_i(t) + \boldsymbol{a}_i(t), & \phi_i(t) = 1 \\ \boldsymbol{p}_i(t), & otherwise, \end{cases} \tag{5}$$

making the UAV update only if active.

A communication quota of each UAV $\mathbf{U}_i(t) \in \{0, 1\}$ indicates communication is either incomplete or complete, which can only be achieved through at least one charge. Note that the predefined goal of a communication quota of $\mathbf{U}_C \in \mathbb{N}$ satisfies the following relation,

$$m \times \boldsymbol{b}_i(0) < \mathbf{U}_C, \tag{6}$$

where $m \in \mathbb{N}$ is used to divide a mission time slot into m communication time slots (see Section 2.3). If a UAV takes off, $\mathbf{U}_i(t)$ is incomplete, and $\mathbf{U}_i(t)$ is complete when the cumulative number of communications of the agent reaches $\mathbf{U}_C$.

The evolution of the operational status $\boldsymbol{\phi}_i(t)$ of each UAV is given by

$$\boldsymbol{\phi}_i(t+1) = \begin{cases} 0, & \mathbf{U}_i(t+1) = 1 \\ & \vee\ \boldsymbol{\phi}_i(t) = 0 \\ & \vee\ b_i(t+1) = 0 \\ 1, & otherwise, \end{cases} \tag{7}$$

where if the UAV takes off, $\boldsymbol{\phi}_i(t)$ is always active, and the operational status $\boldsymbol{\phi}_i(t)$ is inactivated when the communication quota $\mathbf{U}_i(t)$ is completed or the battery energy is depleted.

The ith UAV's battery energy level is updated according to

$$\boldsymbol{b}_i(t+1) = \begin{cases} \boldsymbol{b}_i(t) - 1, & \phi_i(t) = 1\ \wedge\ \mathbf{C}_i(t) = 0 \\ \boldsymbol{b}_i(t) + 10, & \phi_i(t) = 1\ \wedge\ \mathbf{C}_i(t) = 1 \\ \boldsymbol{b}_i(t), & otherwise, \end{cases} \tag{8}$$

where the battery energy level of each UAV $\boldsymbol{b}_i(t)$ is continuously decreased by 1 at each time slot when the operational status is active. Since the battery continues to decrease by one according to the time slot, the remaining battery capacity equals the remaining flying time. UAVs can charge their battery energy level by ten at each time slot when charging.

2.3. Communication channel model

To describe the communications between the UAVs and the sensor nodes, we introduce the concept of a communication time slot $n \in [0, \boldsymbol{N}]$ with $\boldsymbol{N} = m\boldsymbol{T}$, which is different from the mission time slot. Each mission time slot is divided into enough communication time slots so that the position and channel gain of the UAVs within one communication time slot can be assumed to be constant. One communication time slot n has length of $\delta_n = \delta_t/m$ min.

The kth sensor node is located with $\mathbf{N}_k = \left[x_k, y_k, 0\right]^{\mathbf{T}} \in \mathbb{R}^3$ with $k \in \mathrm{K}$ where K is the total number of sensor nodes, which have stable locations during the entire mission time slot $\boldsymbol{T}$ and the same distance between sensor nodes with fixed coordinates. At the start of the mission, a certain range of random data $\mathbf{D}_{k,init}$ is assigned to the kth sensor node, and as information continues to generate data after the start of the mission, we define that the kth sensor node generates new data $\mathbf{D}_{k,new} \sim Poisson(\lambda_p)$ according to the Poisson distribution.

A UAV-to-ground channel model (Bayerlein et al., 2020) is employed to analyze the communications between the UAVs and the sensor nodes. The maximum achievable data rate in communication time slot n for the kth sensor node is defined as

$$\boldsymbol{R}_{i,k}(n) = \delta_n \log_2(1 + \mathrm{SNR}_{i,k}(n)). \tag{9}$$

Note that the length of a communication time slot is multiplied so that the maximum achievable data rate within a mission time slot is constant regardless of the number of communications.

We have

$$\mathrm{SNR}_{i,k}(n) = \frac{P_{i,k}}{\sigma^2} \cdot d_{i,k}^{-\alpha_e} \cdot 10^{\eta_e/10}, \tag{10}$$

where P_k is the transmit power of sensor node k, σ_e^2 is white Gaussian noise power at the receiver, $d_{i,k}$ is the distance between UAV i and sensor node k, α_e is the path loss exponent and $\eta_e \sim \boldsymbol{N}(0, \sigma_e^2)$ is a Gaussian random variable. We assume Line of Sight (LOS) point-to-point communication, and propagation parameter $e \in \{\mathrm{LoS}\}$ is defined.

For communication between the sensor nodes and a UAV, we use the time-division multiple access (TDMA) model as a multiple access protocol. To utilize the models we defined earlier, three significant assumptions have been made, without loss of generality. First, it is assumed that the communication channel between a UAV and the sensor nodes works as an orthogonal resource block with the communication channel between the other UAVs and the sensor nodes. Hence, there is no inter-UAV interference. Also, assuming that sensor nodes are in multi-band mode, a sensor node can communicate with multiple UAVs simultaneously, which means that scheduling decisions can be ignored when determining the action space of the UAVs. Lastly, assuming UAVs always communicate according to the maximum rate, the sensor node only communicates with at most one UAV in a communication time slot. Based on the maximum rate constraint, the scheduling variable $q_{i,k}(n) \in \{0,1\}$ is chosen for all UAVs and all sensor nodes in communication time slot n, in which $q_{i,k}(n)$ is equal to 1 only for the highest $\mathrm{SNR}_{i,k}(n)$.

Then, we can define the kth sensor node's residual data within communication time slot n by

$$d_k(n+1) = d_k(n) - \sum_{i=1}^{\mathrm{I}} q_{i,k}(n) R_{i,k}(n) \delta_n, \tag{11}$$

where I means the total number of agents. The residual data $\mathbf{D}_k(t)$ of the kth sensor node at mission time slot t is

$$\mathbf{D}_k(t) = \begin{cases} \mathbf{D}_{k,init}, & t = 0 \\ d_k(n_t) + \mathbf{D}_{k,new}, & otherwise, \end{cases} \tag{12}$$

where n_t is the communication time slot at mission time slot t. We also assume that new data $\mathbf{D}_{k,new}$ is generated at each mission time slot not each communication time slot.

3. Markov decision process (DEC-POMDP)

The aforementioned operation for the UAV agents and sensor nodes is analyzed as a decentralized partially observable Markov decision process (Dec-POMDP) (Oliehoek and Amato, 2016), which is defined as a septuple $(\mathbb{S}, \mathbb{A}, T,\ \mathbb{O}, O, R, \gamma)$ for practical application in RL. $\mathbb{S}$

stands for the state space, $\mathbb{A}$ is the joint action space, and T contains the transition probability functions. $\mathbb{O}$ means the joint observation space, O is the observation probability function that converts state information into unique observations for one agent. R contains the reward functions. The discount factor γ determines whether to weigh long-term or short-term rewards.

3.1. State space

The state space of the multi-agent problem consists of environmental information, the states of the agents, and the states of the sensor nodes. It is defined as

$$\mathbf{S} = \underbrace{\mathbf{L}}_{\substack{Start\ and\\ Charging\ zone}} \quad \} Environment$$
$$\times \underbrace{\mathbb{R}^{\mathrm{I}\times 3}}_{\substack{UAV\\ Positions}} \times \underbrace{\mathbb{N}^{\mathrm{I}}}_{\substack{Flying\\ Times}} \times \underbrace{\mathbb{B}^{\mathrm{I}}}_{\substack{Operational\\ Status}} \quad \} UAV\ Agents$$
$$\times \underbrace{\mathbb{R}^{\mathrm{K}\times 3}}_{\substack{Sensor\ Node\\ Positions}} \times \underbrace{\mathbb{R}^{\mathrm{K}}}_{\substack{Sensor\ Node\\ Residual\ Data}}. \quad \} Sensor\ Nodes$$

The elements $s_i(t) \in \mathbf{S}$ are

$$s_i(t) = (\mathbf{H}_{layer}, \{\boldsymbol{p}_i(t)\}, \{\boldsymbol{b}_i(t)\}, \{\boldsymbol{\phi}_i(t)\}, \{\mathbf{N}_k\}, \{\mathbf{D}_k(t)\}),\ \forall i \in \mathbb{I},\ \forall k \in \mathbb{K}. \tag{13}$$

$\mathbf{H}_{layer} \in \mathbb{B}^{H\times H\times 1}$ is the tensor layer of the start and charging zone $\mathbf{L}$. The other parts of the septuple are positions, remaining flying times, the operational status of all agents, and positions and residual data that can be collected from all sensor nodes.

3.2. Safety controller

A safety controller is introduced to control the UAV agents so as to operate within the boundaries of the grid world and avoid collisions with each other (Bayerlein et al., 2021). The safety controller alerts agents to cancel a planned action and to hover in the prior position when the agent's position in the next time slot reaches the boundaries of the grid world or is located at the same position as other agents. The actual action to be executed by the ith agent through the safety controller is called a safety action $\boldsymbol{a}_i^s(t)$.

3.3. Reward function

The operational order of the agents is as follows: advance each agent simultaneously, resolve their boundary counters or collisions with each agent, and consume their energy levels. As a result, the agents decide whether to charge their energy or communicate with the sensor node with the best data rate. Lastly, new data is generated for each sensor node. Each of these transitions will have a reward associated with each agent. The reward function is made up of the following elements:

$$r_i(t) = \alpha \sum_{k\in\mathrm{K}} \left(\mathbf{D}_k(t+1) - \mathbf{D}_k(t)\right) + \beta_i(t) + \nu_i(t) + c_i(t) + \omega_i(t) + \zeta. \tag{14}$$

The first term of the sum is a collective reward for all agents' collected data from all sensor nodes within a mission time slot t. It is weighted through the data collection multiplier α. The second term is an individual penalty when agents collide with the boundaries of the grid world or other agents, and is

$$\beta_i(t) = \begin{cases} \beta, & \boldsymbol{a}_i(t) \neq \boldsymbol{a}_i^s(t) \\ 0, & otherwise, \end{cases} \tag{15}$$

where the penalty is characterized through the safety penalty value β. The third term $\nu_i(t)$ is the individual penalty for not fully communicating until it runs out of energy, and is given by

$$\nu_i(t) = \begin{cases} \nu, & \boldsymbol{b}_i(t+1) = 0 \ \wedge\ \mathbf{U}_i(t+1) = 0 \\ 0, & otherwise, \end{cases} \tag{16}$$

The fourth term is a small positive reward while charging:

$$c_i(t) = \begin{cases} c, & \mathbf{C}_i(t+1) = 1 \ \wedge\ \boldsymbol{\phi}_i(t+1) = 1 \\ 0, & otherwise, \end{cases} \tag{17}$$

where the reward c is utilized to encourage charging. The fifth term is a positive reward given when the communication quota is fulfilled:

$$\omega_i(t) = \begin{cases} \omega, & \mathbf{U}_i(t+1) = 1 \\ 0, & otherwise, \end{cases} \tag{18}$$

parametrized through the reward ω to encourage charging. We found that simply imposing a large negative penalty ν when agents run out of their energy is not enough to balance the two main objectives, data collecting and charging. We were able to achieve two objectives in a balanced way with the small positive reward c given while charging and the relatively large positive reward ω given when the communication quota was fulfilled as well as transfer learning. The last term ζ is a constant movement penalty, which induces the agents to find more efficient trajectories.

3.4. Observation space

We need to change the state space $\mathbf{S}$ to the observation space for POMDP. As shown in Theile et al. (2021), we define an observation space for sensor node data, UAV flying times, and operational status through map processing algorithms that consist of map centering, which is the first step, and global-local map processing, which is the second step. Map centering is to place the agent in the center of the map to improve the agent's learning performance. Although there is a disadvantage of increasing the size of the neural networks to be trained with several trainable parameters, there is an advantage that the agent's action can be planned only based on the relative position to the features, e.g., the distances to the start and charging zones, according to the agent's current position. After map centering, there are relatively distant features for which the agents should consider comprehensive path determination and relatively close features such as collision avoidance for path planning. The second step is to ignore some of the details of the distant features, significantly reducing the size of the neural network to be trained while maintaining the training performance, as shown in Theile et al. (2021).

Mapping

The map centering and global-local algorithms can be used after the state spaces are stacked in the form of a map layer that projects the relative features according to the agent's position. According to the positions of the agents and sensor nodes, the mapping is performed in pairs as follows:

$$(\{\boldsymbol{p}_i(t)\}, \{\boldsymbol{b}_i(t)\}), (\{\boldsymbol{p}_i(t)\}, \{\boldsymbol{\phi}_i(t)\}), (\{\mathbf{N}_k\}, \{\mathbf{D}_k(t)\}),$$

where the size of each mapped map layer is $\mathbf{A}_{layer} \in \mathbb{R}^{M\times M}$ and the map layer $\mathbf{A}_{layer}$ can be stacked as a tensor of $\mathbb{R}^{M\times M\times n}$ when each layer has the same type.

Map centering and global-local map

The map layer $\mathbf{A}_{layer} \in \mathbb{R}^{M\times M\times n}$ is converted into a centered tensor $\mathbf{B}_{layer} \in \mathbb{R}^{M_c\times M_c\times n}$ with $M_c = 2M - 1$. In the centered tensor $\mathbf{B}_{layer}$, the original tensor $\mathbf{A}_{layer}$ is moved so that position of the agent is centered, and the rest of the extended part of the centered tensor $\mathbf{B}_{layer}$ is filled with the padding value $\mathbf{x}_{\mathbf{pad}} \in \mathbb{R}$.

The tensor $\mathbf{B}_{layer} \in \mathbb{R}^{M_c\times M_c\times n}$ is used in two ways. In order to create a local map, the centered tensor $\mathbf{B}_{layer}$ is cropped with cropping parameter l. The local tensor $\mathbf{C}_{layer}$, which is the expression of the local map, appears as $\mathbf{C}_{layer} \in \mathbb{R}^{l\times l\times n}$. For the global tensor $\mathbf{D}_{layer}$, which is the expression of the global map, we use an average pooling algorithm with pooling cell size g to ignore the detailed level of distant features. The global tensor $\mathbf{D}_{layer}$ is denoted by $\mathbf{D}_{layer} \in \mathbb{R}^{\left\lfloor \frac{M_c}{g} \right\rfloor \times \left\lfloor \frac{M_c}{g} \right\rfloor \times n}$. As l increases, the size of the local map increases, while as g increases, the pooling cell size increases, reducing the size of the global map.

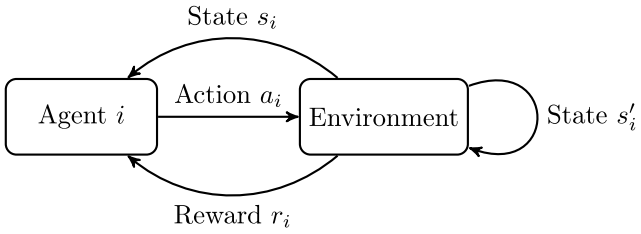


Fig. 2. The basic framework of RL.

Observation space Ω

Through the map processing, the observation space $\boldsymbol{\Omega}$, which is the input space for the agent, is defined as

$$\boldsymbol{\Omega} = \underbrace{\boldsymbol{\Omega}_l}_{\text{Local Map}} \times \underbrace{\boldsymbol{\Omega}_g}_{\text{Global Map}} \times \underbrace{\mathbb{N}}_{\text{Flying Time}}, \tag{19}$$

where the local map is

$$\boldsymbol{\Omega}_l = \underbrace{\mathbb{B}^{l\times l\times 1}}_{\substack{\textit{Local}\\ \textit{Environment}}} \times \underbrace{\mathbb{R}^{l\times l}}_{\substack{\textit{Local}\\ \textit{Data}}} \times \underbrace{\mathbb{N}^{l\times l}}_{\substack{\textit{Local}\\ \textit{Flying time}}} \times \underbrace{\mathbb{B}^{l\times l}}_{\substack{\textit{Local}\\ \textit{Operat'l status}}}, \tag{20}$$

and the global map is

$$\boldsymbol{\Omega}_g = \underbrace{\mathbb{R}^{\tilde{g}\times\tilde{g}\times 1}}_{\substack{\textit{Global}\\ \textit{Environment}}} \times \underbrace{\mathbb{R}^{\tilde{g}\times\tilde{g}}}_{\substack{\textit{Global}\\ \textit{Data}}} \times \underbrace{\mathbb{R}^{\tilde{g}\times\tilde{g}}}_{\substack{\textit{Global}\\ \textit{Flying time}}} \times \underbrace{\mathbb{R}^{\tilde{g}\times\tilde{g}}}_{\substack{\textit{Global}\\ \textit{Operat'l status}}}, \tag{21}$$

with $\tilde{g} = \left\lfloor \frac{M_c}{g} \right\rfloor$. Observation space $o_i(t) \in \boldsymbol{\Omega}$ is defined as a tuple

$$\begin{aligned} o_i(t) = (&\mathbf{M}_{l,i}(t), \mathbf{D}_{l,i}(t), \mathbf{B}_{l,i}(t), \boldsymbol{\Phi}_{l,i}(t),\\ &\mathbf{M}_{g,i}(t), \mathbf{D}_{g,i}(t), \mathbf{B}_{g,i}(t), \boldsymbol{\Phi}_{g,i}(t), b_i(t)). \end{aligned} \tag{22}$$

In an observation space tuple, $\mathbf{M}_{l,i}(t)$ is the local observation of agent i of the environment, $\mathbf{D}_{l,i}(t)$ is the local observation of the residual data, $\mathbf{B}_{l,i}(t)$ is the local observation of the remaining flying time, and $\boldsymbol{\Phi}_{l,i}(t)$ is the local observation of the operational status. $\mathbf{M}_{g,i}(t), \mathbf{D}_{g,i}(t), \mathbf{B}_{g,i}(t)$, and $\boldsymbol{\Phi}_{g,i}(t)$ are the respective global observations. Note that surplus elements of the remaining flying time allow the agents to understand their remaining flying time better.

As a result, we artificially transformed the POMDP to solve the operational system problem we defined, which significantly reduces the neural network's size as proved in Theile et al. (2021), making the training procedure feasible and reducing the training time.

4. Multi-Agent Reinforcement Learning (MARL) and transfer learning

Now we can formulate the RL as Dec-POMDP is depicted (see Section 3.4). RL is a policy optimization process that selects an action a_i based on a policy π in the current state s_i and receives feedback called a reward r_i through interaction with the environment. At the same time, it changes to the next state s'_i. The basic framework for RL is shown in Fig. 2. Note that, because of the way we defined Dec-POMDP, the state that becomes the input space for the agent is an observation state o_i observed by the agent, not the actual state s_i.

4.1. Double deep Q-learning

One of the promising algorithms in RL is Q-learning (Sutton and Barto, 2018), which maximizes the cumulative discounted reward by repeatedly carrying out the action a_i in state s_i until the end of the episode. It is given by

$$Q(s_i, a_i) = r(s_i, a_i) + \gamma_q \sum_{s'_i \in S} p(s_i, a_i, s'_i) Q^*(s'_i, a'_i). \tag{23}$$

Note that s_i, a_i is the same as $s_i(t)$, $a_i(t)$ as well as s'_i, a'_i is the same as $s_i(t+1)$, $a_i(t+1)$ for ease of exposition. In (23), $r(s_i, a_i)$ represents the immediate reward given to action a_i in state s_i, $\gamma_q \in [0, 1]$ is the discount factor, $p(s_i, a_i, s'_i)$ is the transition probability to s'_i after action a_i in the present state s_i, and $Q^*(s'_i, a'_i) = \max Q(s'_i, a'_i)$ represents the optimal Q-value of action a'_i in next state s'_i. The Q-value, which is the expected value of the sum of future rewards given the state s_i and the action a_i, is updated by the following formula:

$$\begin{aligned} Q(s_i, a_i) \leftarrow (1-\alpha_q) Q(s_i, a_i) + \alpha_q \times \\ \left[r(s_i, a_i) + \gamma_q \sum_{s'_i \in S} p(s_i, a_i, s'_i) Q^*(s'_i, a'_i) \right], \end{aligned} \tag{24}$$

where $\alpha_q \in (0, 1]$ is the learning rate. As such, the optimal policy π_* is derived from

$$\pi_*(s_i) = \underset{a_i \in A}{argmax}\ Q(s_i, a_i). \tag{25}$$

Q-learning is performed using a Q-table, where the rows and columns consist of the action and state to calculate the Q-value. Q-learning is computationally effective when the environment is simple. But when the environment becomes more complex with a highly extensible state space, it has difficulty converging because the Q-table grows exponentially.

DQN was introduced to approximate the Q-table using deep neural networks with weights θ (Mnih et al., 2015). The advantage of using these approximations is that DQN will converge in complex environments, wherein neural networks show significant data efficiency in generalization with experience replay and the target network. Experience replay allows networks to store transitions (s_i, a_i, r_i, s'_i) in the experience replay memory $\mathbf{D}$. During training, instead of deploying only the current transition (s_i, a_i, r_i, s'_i), mini-batches of transitions are chosen from the replay memory and used to train the network, which leads to reducing the correlations in the sequence of training transitions as well as preventing being driven to a local minimum. The target network is a separate network that duplicates the original network with weights θ^-. This network is updated at slower intervals than the original network to stabilize the overall convergence of the network during training. Then, the Q-network updates its weights to minimize a loss function represented by the mean square error as such.

$$L_i(\theta) = \mathbb{E}_{s_i, a_i, r_i, s'_i \sim \mathbf{D}} \left[\| y_i^{DQN} - Q_i(s_i, a_i; \theta) \|^2 \right], \tag{26}$$

where the target value is

$$y_i^{DQN} = r_i(s_i, a_i) + \gamma_q \max_{a'_i \in A_i} Q_i(s'_i, a'_i; \theta^-). \tag{27}$$

The parameters of the target network θ^- are updated either simply by means of a periodic copy of θ for each episode or a soft update for each state transition after each update of θ. As a method to update the parameters of the target network θ^-, we used

$$\theta^- \leftarrow \theta\tau + \theta^-(1-\tau), \tag{28}$$

where $\tau \in [0, 1]$ is an updating factor that adjusts the adaptation speed and we choose τ to be 0.005 (Bayerlein et al., 2021).

The target value y_i^{DQN} is inevitably different from the optimal value Q^* because the action of the next state a'_i is selected with the target network parameter θ^- and then updated to the max value of

$Q_i(s_i', a_i'; \theta^-)$ with the same target network parameter θ^-. The overestimation problem occurs because the max calculation is performed twice with the same parameter θ^-. Double-deep Q networks (DDQN), which separate the parameters of the Q network when calculating the target value y_i^{DDQN}, was introduced in Van Hasselt et al. (2016). Finally, the target value y_i^{DDQN} is defined as

$$y_i^{DDQN} = r_i(s_i, a_i) + \gamma_q Q_i\left(s_i', \underset{a_i' \in A_i}{\text{argmax}} Q_i(s_i', a_i'; \theta); \theta^-\right). \quad (29)$$

4.2. Multi-agent deep RL (MADRL) and transfer learning

There are a few approaches to address the challenge of optimizing each agent's policy control while considering the other agents in a multi-agent environment. The best way to apply our learning approach is decentralized deployment with knowledge transfer (Chen, 2019). With this approach, agent execution occurs individually, which is the decentralized selection of the policy. We chose all agents to train and learn their policies on the same network and share parameters with the other agents. Each agent's experience acquired through decentralized deployment is added to a common replay memory. During training, all the agent's experiences are optimized as mini-batches are chosen from the replay memory, which leads to faster convergence.

As defined in (14), our agents cannot be characterized as fully cooperative because the central element of the reward function, which has the same structure for each agent and is calculated with decentralized execution, is based on charging and jointly collected data (Zhang et al., 2021). Our setting is a simple cooperative where all the agents learn the optimal cooperation policy.

Balancing the multiple objectives of data collection and charging can be much closer to having practical applications to the real world (Dulac-Arnold et al., 2019). We found that simply adding individual components to the reward function was insufficient to achieve the two main objectives effectively. The advantage of transfer learning for agents is employed to learn how to balance multiple objectives effectively (Punma, Unpublished results, 0000). When there is no transfer learning in the first place, agents continue to collect data to receive positive rewards even if they run down their batteries, which will impose a sizeable negative reward in the near future. We use transfer learning to get the agent to learn their policy divided into two stages. In the first training, we set agents to not take into account running down their battery, by removing the continuous battery decrease according to the mission time slot. In the next training, we continuously decrease their battery according to the mission time slot, while keeping all other settings the same. Through transfer learning, agents could balance the two objectives by charging when needed.

The detailed procedure for the transfer learning part of our algorithm is presented in Algorithm 1. The reason why we test episodes in lines 10–11 is that since various scenario parameters are included together in the training process of one control policy, making only a few evaluations may reduce the reliability of the evaluation due to the presence of extreme cases. Therefore, a moving average of periodic multiple evaluations and the model weight separately when this updated value shows the best performance are accumulated. The weights are loaded during the second training phase or evaluation phase.

4.3. Complexity analysis

During the first and second training phases, the DRL agent observes the environment where the states of POMDP are defined as input for the DRL algorithm in Section 3.4. The agent utilizes its trained network to carry out an action $\boldsymbol{a}_i^s(t)$ which represents an actual direction. The total computational complexity for the fully connected layers is computed as the number of multiplications (Liu et al., 2019): $O(\sum_{f=1}^{F} n_f \cdot n_{f-1})$, where n_f is the number of neurons in fully-connected layer f.

Algorithm 1 Training and Evaluation process based on transfer learning for the multiple objectives problem.

1: Initialize episode parameters.
2: Initialize experience replay memory **D**.
3: Initialize the weights of original network θ and that of target network θ^-.
4: Fill replay memory with random action.
5: **Step-I: First training phase**
6: Set ϵ as 0.1.
7: Set decrement of the energy level of agent per step as 0.
8: **for** *epoch* $ep = 0, 1, \ldots$ **do**
9: Train episode.
10: **if** $ep\%5 = 0$ **then**
11: Test episode.
12: **end if**
13: **if** the best performance is achieved **then**
14: Save the model weights as θ'.
15: **end if**
16: **end for**
17: **Step-II: Second training phase**
18: Set ϵ to ranging from 0.5 to 0.01.
19: Set decrement of the energy level of agent per step as -1.
20: Load the model weights as θ'.
21: **Repeat** 8 – 16.
22: **Step-III: Evaluation phase**
23: Load the model weights as θ'.
24: Hard update the target model weights as θ'.
25: **for** *epoch* $ep = 0, 1, \ldots,$ 1000 **do**
26: Test episode.
27: **end for**

Table 2
Value ranges of the scenario parameters.

Scenario parameters	Min	Max
Number of deployed UAVs	1	3
Initial data volume to be collected	5.0	20.0
Maximum flying time	70	80
Charging start capacity	10%	50%
Charging complete capacity	60%	100%
Number of sensor nodes	5	25

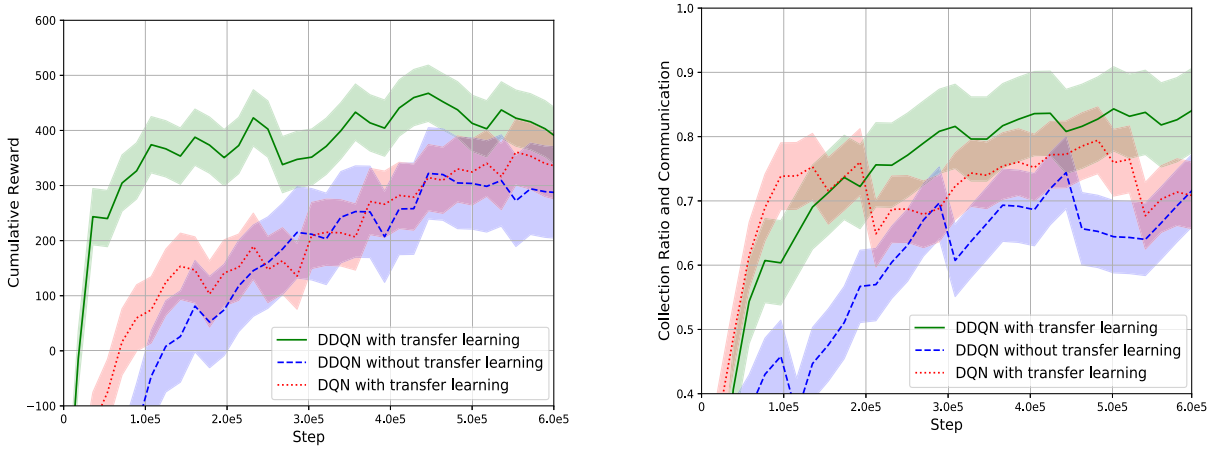
5. Simulations

5.1. Settings

The simulation was implemented in Tensorflow 2.4.0 with a NVIDIA Quadro RTX 4000 Graphics Processor and a 128 GB RAM. This UAV-oriented data collection system was evaluated on a map that is discretized into 32×32 cells. At the beginning of each episode, scenario parameters that determine the scenario are sampled randomly within a predetermined range. The scenario parameters are as follows: number of deployed UAVs, the initial amount of data to be collected from sensor nodes, flying time available for UAVs at the start of each episode, charging start capacity, charging complete capacity, number of sensor nodes. The exact ranges of values from which the randomized scenario parameters can be taken in Table 2.

The grid cell size c is 800 m, and the constant altitude h at which the UAVs fly is 10 m. Each mission time slot contains four scheduled communication time slots, that is, $m = 4$. Propagation parameters are chosen in line with European Telecommunications Standards Institute (ETSI) (2017), the rural macro scenario, with $\alpha_{LoS} = 2.27$ and $\sigma^2_{LoS} = 2$.

The network architecture of the proposed algorithm consists of convolution layers that accommodate $\mathbf{C}_{layer}$ and $\mathbf{D}_{layer}$ as inputs with ReLU activation that have a flattened layer that has three fully connected hidden layers containing 256 neurons respectively and a fully



(a) Cumulative reward per episode

(b) Data collection with communication fulfillment per episode

Fig. 3. Comparisons of the training processes among the proposed algorithm and baseline algorithms. These curves include the average and 99% confidence interval line shown through the exponential moving average. Note that the metrics are displayed with training steps equivalent to the number of epochs while training episode length is variable.

Table 3
Event reward structure.

Symbol	Event	Reward
α	Data collection multiplier	+1.3
β	Collisions with other agents or boundaries	−1.0
γ	Run out of energy	−300.0
δ	While charging	+0.05
ϵ	Communication quota is fulfilled	+100.0
ζ	Standard movement	−0.1

connected layer of which the size of the output is $\mathbf{A}$. The Q-values from the last fully connected layer are used as an input observation through the argument maximization policy while evaluating the agent and the softmax policy when training the agent.

The environmental settings to implement the sensor nodes are as follows. Since it is a sustainable system using charging, one episode is set to end when the UAV energy level becomes empty or when a communication quota is fulfilled. We set the predefined communication quota $\mathbf{U}_C$ as 440. The new data $\mathbf{D}_{k,new}$ allocated for each mission time slot to each sensor node follows a Poisson distribution, where $\lambda_p = 0.02$. The sensor nodes are randomly scattered 5 to 25 different points for each episode from the combination of x-axis coordinates (3, 8, 13, 18, 23) and y-coordinates (6, 11, 16, 21, 26). The values of the exact parameters, as they appeared in (14), are shown in Table 3.

We use the following metrics to evaluate the agent's performance under different scenario instances. First, *communication fulfillment* is the ratio of whether all agents have fulfilled their communication quotas at the end of an episode. Second, *collection ratio* is the ratio of data collected until the end of the episode to the total data, which contains the data given to the sensor nodes at the beginning of the episode and the continuously increasing data. Lastly, *collection ratio and communication* is the product of *communication fulfillment* and *collection ratio* per episode.

5.2. DRL path planning based on transfer learning

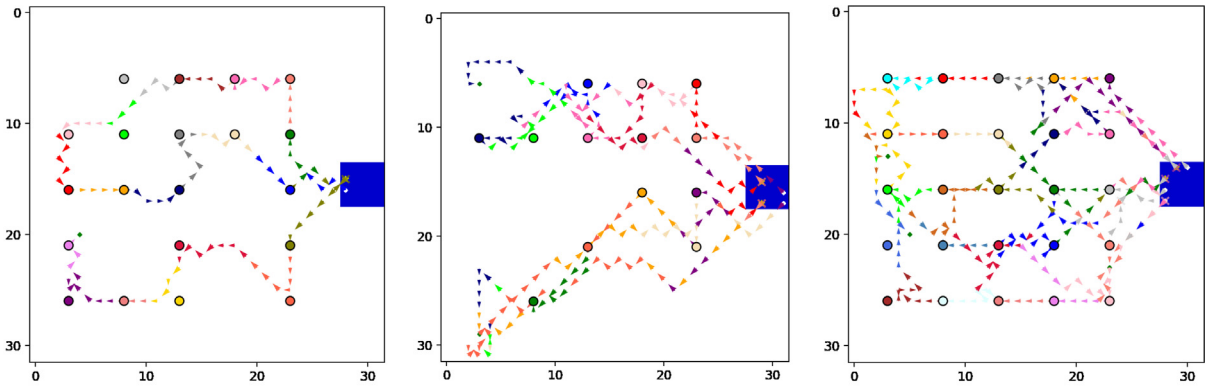
To demonstrate that transfer learning and DDQN are essential to achieving higher training performance, we compare the proposed algorithm with two baseline algorithms. We first compare training processes with and without transfer learning. When applying transfer learning, ϵ is tuned with e-greedy exploration starting at 0.5 and decaying to 0.01 in the second stage, as shown in Algorithm 1. The reason for using decaying e-greedy exploration is that it was possible to meet the multiple objectives of charging and data collection experimentally. Furthermore, we compare the proposed algorithm with a conventional algorithm, the DQN algorithm. The DQN algorithm applies transfer learning in the same way as our proposed algorithm.

Fig. 3 depicts the cumulative reward and the collection ratio with communication fulfillment metrics over the training step. These results show performance improvements when the proposed algorithm is used. In Fig. 3a, we can see that the proposed algorithm-based agent performs better in the entire training steps. In Fig. 3b, it can be observed that a curve to which the proposed algorithm is applied shows better performance in the latter training steps. We could observe that the curves to which the DQN baseline algorithm was applied already converge from training step 1.0e5, whereas the curve to which the proposed algorithm was applied outperforms in training step 2.0e5. This is because a proposed algorithm-based agent has already been trained to collect data, so the performance has risen more steeply while training communication fulfillment through charging. This is significant in that the performance when the proposed algorithm is applied to both metrics is better.

5.3. Visualization of simulation results

The scenario is presented based on the scenario parameter set in *Settings* (see Section 5.1.). Performance in the scenario is evaluated using Monte Carlo simulations over the defined full range of scenario parameters. The overall average performance is shown in Table 4. 5 *actions* is the result of defining action space as the $\mathbf{A} = \{North, East, South, West, Hover\}$. On the other hand, 9 *actions* is the action space that we defined in Section 2.2. As the agents' action space expanded, the performance of our proposed algorithm improved significantly. Given that successful communication can be achieved through at least one charge, the agents show good communication fulfillment performance. We believe that the collection ratio cannot be 100%, because there are random scenario parameters that include flying time, the number of deployed UAVs or sensor nodes, and charging parameters, and an extreme combination of these could be selected, e.g., the number of deployed UAVs could be one, or it could be set that charging starts when the remaining capacity is 10%, or new data could be incremented for each sensor node until the end of the episode.

In Fig. 4, three example scenarios describe how the path planning was adapted as the number of deployed UAVs increased from 1 to 3. All other scenario parameters remain fixed at the values described in Fig. 4. The fixed scenario parameter values were set as the average value of the defined full range of the scenario parameters.



(a) I = 1, communication fulfillment 96.6%, data collection ratio 73.6%

(b) I = 2, communication fulfillment 93.3%, data collection ratio 89.4%

(c) I = 3, communication fulfillment 93.0%, data collection ratio 93.1%

Fig. 4. Example trajectories for 32 × 32 map with charging start capacity = 30%, charging complete capacity = 80%, all sensor node with initial data volume = 12.5 and a maximum flying time = 80 steps.

Table 4
Performance metrics averaged over 1000 random scenario Monte Carlo iterations.

Metric	5 *actions*	9 *actions*
Communication fulfillment	90.1%	90.0%
Collection ratio	79.8%	90.8%
Collection ratio and communication	71.9%	81.5%

Fig. 4a shows a single agent trying to collect as much data as possible with one-time charging. The agent collects data while flying over the entire range of the sensor nodes. We can observe that a single agent has no choice but to ignore a few nodes because the sensor nodes are too widely deployed for all of them to be covered. Interesting points can be seen in Fig. 4b, where a second UAV is assigned. Unlike Fig. 4a, it can be seen that two agents adapt the path planning to divide the entire area covered by sensor nodes into two sectors, which can be confirmed by the dramatic increase in the data collection ratio from 73.6% to 89.4% as well as the agents can reach each sensor node at least two times. It can be seen that communication fulfillment has declined from 96.6% to 93.3% as one more UAV is assigned. In Fig. 4c, with the addition of the third UAV, the data collection ratio increased to 93.1%. However, communication fulfillment decreased slightly due to the large number of agents.

5.4. Effect of scenario parameters on performance and macro-level benefits

What we want to emphasize is the effective reduction of the training time. As demonstrated in Bayerlein et al. (2021) and Theile et al. (2021), the global-local map approach has reduced the required size of the network enough for feasible learning. Also, training together with various scenario parameters helps find the sweet spot for each scenario. For example, in Fig. 4, it can be seen that Fig. 4c is the scenario closest to the sweet spot when deciding how many UAVs should be deployed with the other scenario parameters given. The performance for specific parameters can be derived using the advantage of our approach derived from learning the generalized path planning policy for various scenario parameters.

Fig. 5 shows the effect on three metrics for each specific scenario parameter. As demonstrated for the example trajectories shown in Fig. 4, Fig. 5 shows that the performance is the best when the number of agents is three. As the number of agents increases further, we observed that the constantly generated data were collected well. In Fig. 5b, it can be seen that the communication fulfillment is not lower than other charging start capacities even when the charging start capacity, which can be evaluated as adventurous in data collection, is 10%–20%. This is because the agents have learned enough within the defined range of scenario parameters. Similarly, in Fig. 5c, it can be seen that the agents have learned well about all cases within the range of scenario parameters. As shown in Fig. 5d, it is beneficial to allocate a large amount of initial data per sensor node up to a range of [12.5, 15.0], but it is negative that the initial data is allocated over the range. Based on the reward structure (14), this is because there is less data to collect before [12.5, 15.0], and after the range, the collection ratio decreased because the data per sensor node becomes abundant. Fig. 5e indicates that increasing the initial energy level for UAVs has a positive relationship with the collection ratio and communication.

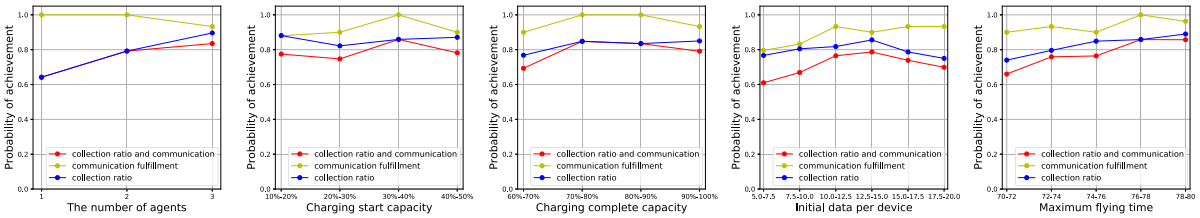
However, we observed that when a large amount of communication quota was given in the multi-agent situation since the size of the charging area is small, 4 × 4 cells, the probability of achieving a communication fulfillment is reduced. In addition, the new data needs to be set larger in λ_p for the agents to collect data continuously. As a result, there is a complex relationship between the UAV flying time, the communication quota, and the amount of new data.

After fixing the parameter value showing the best performance in each scenario parameter range, it was confirmed as a result of 300 Monte Carlo iterations that the collection ratio and communication was 92%, the communication fulfillment was 97%, and the collection ratio was 95%. These values mean that the learning is carried out over the entire range of scenario parameters at once, so we can effectively check each parameter that performs optimally in the environment we defined and how much the best performance is at that time. In other words, sweet spots in the environment can be effectively found at the macro-level.

5.5. Scaling to complicated environments

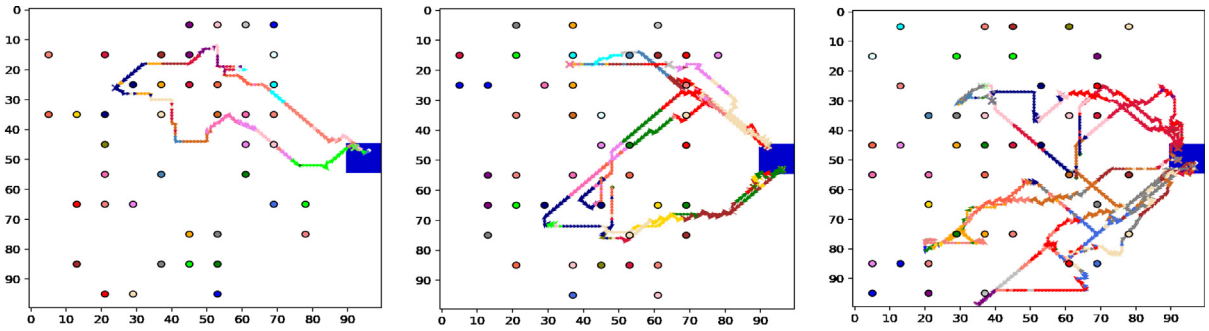
We adopted two more extensive environments to test whether each agent performs effectively even in more general environments. The first map is a 100 × 100 grid in which the number or locations of sensor nodes vary for each episode. The sensor nodes are randomly scattered 36 to 49 for each episode. The maximum flying time is from 200 to 240, and $\mathbf{U}_C$ is 1200. α was changed from 1.3 to 0.2, considering that it became difficult to harvest data due to the increased distance between UAVs and sensor nodes in a 100 × 100 map. Other environmental settings are the same as in Section 5.1. The second map is also a 100 × 100 where grid sensor nodes are randomly scattered 40 to 80 for each episode. The most distinguishable difference from the first map is that a start and charging zone is located in the center of the map. Other environmental settings are the same as the first map.

In Figs. 6 and 7, example trajectories represent how the path planning was adapted as the number of deployed UAVs increased from 1 to 3 in the extended maps. All other scenario parameters remain fixed



(a) The number of agents. (b) Charging start. (c) Charging complete. (d) Initial data per device. (e) Maximum flying time.

Fig. 5. Effect on three metrics according to the scenario parameters. Each data point averages 300 Monte Carlo iterations for each scenario parameter space while maintaining the parameters to be investigated.

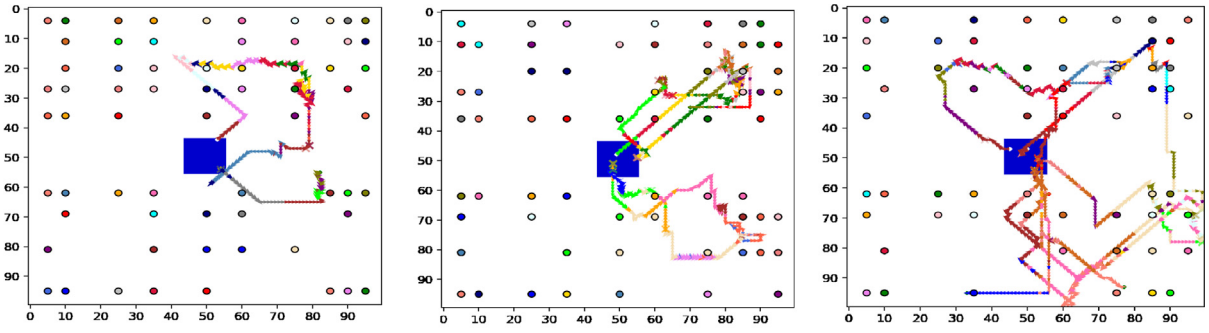


(a) I = 1, communication fulfillment 100.0%, data collection ratio 21.2%

(b) I = 2, communication fulfillment 95.6%, data collection ratio 28.3%

(c) I = 3, communication fulfillment 93.3%, data collection ratio 35.6%

Fig. 6. Example trajectories of the first extended map with charging start capacity = 30%, charging complete capacity = 80%, all sensor node with initial data volume = 12.5, a maximum flying time = 220 steps and the number of sensor nodes = 42.



(a) I = 1, communication fulfillment 100.0%, data collection ratio 18.6%

(b) I = 2, communication fulfillment 88.0%, data collection ratio 28.1%

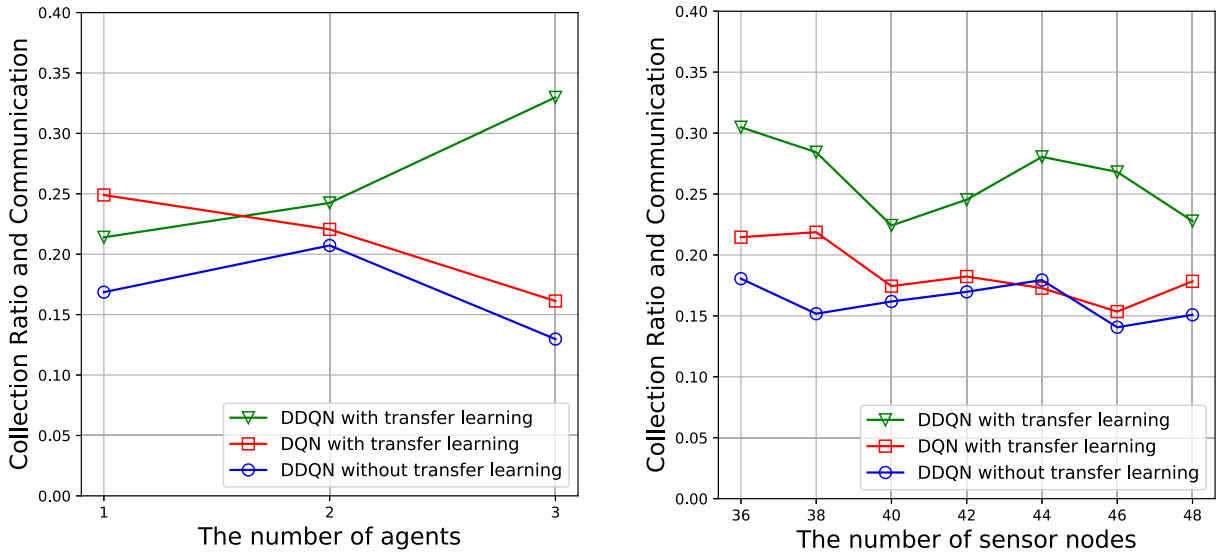
(c) I = 3, communication fulfillment 64.9%, data collection ratio 34.2%

Fig. 7. Example trajectories of the second extended map with charging start capacity = 30%, charging complete capacity = 80%, all sensor node with initial data volume = 12.5, a maximum flying time = 220 steps and the number of sensor nodes = 60.

at the values described in Figs. 6 and 7. The fixed scenario parameter values were set as the average value of the defined full range of the scenario parameters. Analogous to the trend of Fig. 4, Fig. 6 shows that as the number of agents increases, communication fulfillment decreases, and the data collection ratio increases. Also, Fig. 6c can be a scenario closest to the sweet spot when deciding how many UAVs should be deployed with the other scenario parameters given. However, the entire data collection ratio dropped compared to Fig. 4 as the size of the environment and the number of sensor nodes expanded. Fig. 7 also shows a similar trend to Fig. 4, but it is noteworthy that when the number of agents was three, the communication fulfillment decreased significantly to 64.9%. In the fixed scenario parameters in Fig. 7, we could confirm that the sweet spot is when it is two agents.

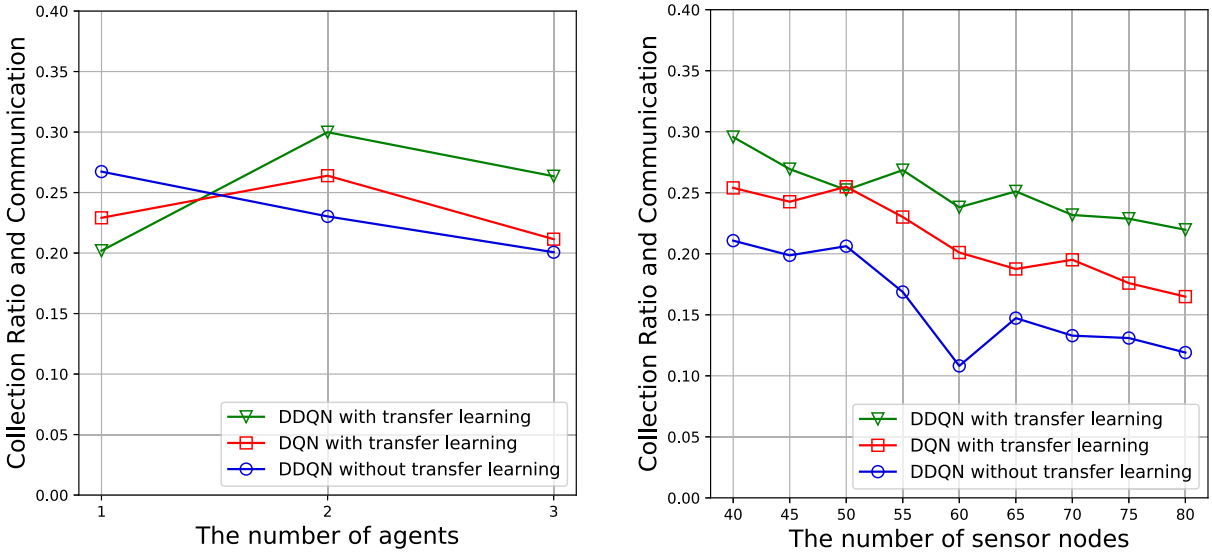
Figs. 8 and 9 show the collection ratio and communication performance with respect to the number of agents and sensor nodes in two extended maps. We found that our proposed algorithm outperforms the baseline algorithms in most cases. Even in expanded situations, there were substantial performance improvements. Fig. 8a shows that as the number of agents increases, the performance difference with the baseline algorithms increases. Regardless of the number of sensor nodes, the proposed scheme maintains outstanding performance above 0.22 in Fig. 8b.

In Fig. 9a, the proposed algorithm showed superior performance with a difference of about 0.05 in the three agents situation except when the number of agents was one. The trend in Fig. 7c, which showed performance degradation when the number of agents was three, was the same when the number of agents increased to three in Fig. 9a. In Fig. 9b, while all the algorithms exhibited performance degradation with the increasing number of sensor nodes, the proposed algorithm showed superior performance on the entire range of the number of sensor nodes. The proposed scheme confirmed performance improvements



(a) Collection Ratio and Communication versus I (b) Collection Ratio and Communication versus K

Fig. 8. Collection ratio and communication versus I and K in first extended map.



(a) Collection Ratio and Communication versus I (b) Collection Ratio and Communication versus K

Fig. 9. Collection ratio and communication versus I and K in second extended map.

when extended to near-continuous environments for this multi-agent system.

6. Conclusion and future work

We proposed a multi-agent deep reinforcement learning approach to build a sustainable system that UAVs can achieve with data harvesting from sensor nodes and charging when necessary. Through DDQN used with various scenario parameters and global-local map processing, it was confirmed that efficient learning was possible for agents in data collection, charging, and path planning constraints. The results show that the agents achieve balanced multiple objectives using a reward function and transfer learning with a decaying e-greedy exploration technique. When a specific environment is defined, it can be seen from the results that it is possible to control the scenario parameters at a macro level to show the best performance.

For future work, the learning objective needs to be extended to collecting data considering the age of information collected from the sensor node in non-grid 3D environments with policy-based algorithms for continuous actions. It could be necessary to learn by including even the altitude of the UAV in the action space using a deep deterministic policy gradient (DDPG) algorithm to implement non-grid UAV movements. In non-grid environments with exponentially large states, effective learning can be carried out with an attention mechanism allowing each agent to learn the essential states by focusing on areas needed to train. It would be considerable to give each agent a unique network that makes an individual policy control operate in a decentralized deployment and decentralized training.

CRediT authorship contribution statement

Mincheol Seong: Writing – original draft, Conceptualization, Investigation. **Ohyun Jo:** Writing – review & editing, Conceptualization,

Funding acquisition. **Kyungseop Shin:** Writing – review & editing, Conceptualization, Funding acquisition.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: MinCheol Seong reports financial support was provided by The National Research Foundation of Korea.

Data availability

The data that has been used is confidential.

Acknowledgments

This work was supported by the National Research Foundation of Korea [grant number 2021R1F1A1064059].

References

Alkandari, A., Alabduljader, Y., Moein, S.M., et al., 2012. Water monitoring system using Wireless Sensor Network (WSN): Case study of Kuwait beaches. In: 2012 Second International Conference on Digital Information Processing and Communications (ICDIPC). IEEE, pp. 10–15. http://dx.doi.org/10.1109/ICDIPC.2012.6257270.

Bayerlein, H., Theile, M., Caccamo, M., Gesbert, D., 2020. UAV path planning for wireless data harvesting: A deep reinforcement learning approach. In: GLOBECOM 2020-2020 IEEE Global Communications Conference. IEEE, pp. 1–6. http://dx.doi.org/10.1109/GLOBECOM42002.2020.9322234.

Bayerlein, H., Theile, M., Caccamo, M., Gesbert, D., 2021. Multi-UAV path planning for wireless data harvesting with deep reinforcement learning. IEEE Open J. Commun. Soc. 2, 1171–1187. http://dx.doi.org/10.1109/OJCOMS.2021.3081996.

Bottarelli, L., Bicego, M., Blum, J., Farinelli, A., 2019. Orienteering-based informative path planning for environmental monitoring. Eng. Appl. Artif. Intell. 77, 46–58. http://dx.doi.org/10.1016/j.engappai.2018.09.015.

Chen, G., 2019. A new framework for multi-agent reinforcement learning–centralized training and exploration with decentralized execution via policy distillation. http://dx.doi.org/10.48550/arXiv.1910.09152, arXiv preprint arXiv:1910.09152.

Dulac-Arnold, G., Mankowitz, D., Hester, T., 2019. Challenges of real-world reinforcement learning. http://dx.doi.org/10.1007/s10994-021-05961-4, arXiv preprint arXiv:1904.12901.

European Telecommunications Standards Institute (ETSI), 2017. Study on channel model for frequencies from 0.5 to 100GHz. (accessed May 2017) https://www.etsi.org/deliver/etsi_tr/138900_138999/138901/14.00.00_60/tr_138901v140000p.pdf.

Fu, S., Tang, Y., Wu, Y., Zhang, N., Gu, H., Chen, C., Liu, M., 2021. Energy-efficient UAV enabled data collection via wireless charging: A reinforcement learning approach. IEEE Internet Things J. http://dx.doi.org/10.1109/JIOT.2021.3051370.

Kishk, M., Bader, A., Alouini, M.-S., 2020. Aerial base station deployment in 6G cellular networks using tethered drones: The mobility and endurance tradeoff. IEEE Veh. Technol. Mag. 15 (4), 103–111. http://dx.doi.org/10.1109/MVT.2020.3017885.

Lahmeri, M.-A., Kishk, M.A., Alouini, M.-S., 2021. Artificial intelligence for UAV-enabled wireless networks: A survey. IEEE Open J. Commun. Soc. 2, 1015–1040. http://dx.doi.org/10.1109/OJCOMS.2021.3075201.

Liu, C.H., Ma, X., Gao, X., Tang, J., 2019. Distributed energy-efficient multi-UAV navigation for long-term communication coverage by deep reinforcement learning. IEEE Trans. Mob. Comput. 19 (6), 1274–1285. http://dx.doi.org/10.1109/TMC.2019.2908171.

Liu, K., Yang, Z., Li, M., Guo, Z., Guo, Y., Hong, F., Yang, X., He, Y., Feng, Y., Liu, Y., 2010. Oceansense: Monitoring the sea with wireless sensor networks. ACM SIGMOBILE Mob. Comput. Commun. Rev. 14 (2), 7–9. http://dx.doi.org/10.1145/1854219.1854223.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al., 2015. Human-level control through deep reinforcement learning. Nature 518 (7540), 529–533. http://dx.doi.org/10.1038/nature14236.

Oliehoek, F.A., Amato, C., 2016. A Concise Introduction To Decentralized POMDPs. Springer-Verlag, Berlin.

Pérez-Carabaza, S., Scherer, J., Rinner, B., López-Orozco, J.A., Besada-Portas, E., 2019. UAV trajectory optimization for Minimum Time Search with communication constraints and collision avoidance. Eng. Appl. Artif. Intell. 85, 357–371. http://dx.doi.org/10.1016/j.engappai.2019.06.002.

Punma, C., 0000. Autonomous Vehicle Fleet Coordination With Deep Reinforcement Learning, Unpublished results.

Shen, S., Han, Y., Wang, X., Wang, Y., 2019. Computation offloading with multiple agents in edge-computing–supported IoT. ACM Trans. Sensor Netw. 16 (1), 1–27. http://dx.doi.org/10.1145/3372025.

Skorobogatov, G., Barrado, C., Salamí, E., 2020. Multiple UAV systems: A survey. Unmanned Syst. 8 (02), 149–169. http://dx.doi.org/10.1142/S2301385020500090.

Sutton, R.S., Barto, A.G., 2018. Reinforcement Learning: An Introduction. MIT Press, Cambridge, MA.

Theile, M., Bayerlein, H., Nai, R., Gesbert, D., Caccamo, M., 2021. UAV path planning using global and local map information with deep reinforcement learning. In: 2021 20th International Conference on Advanced Robotics (ICAR). IEEE, pp. 539–546. http://dx.doi.org/10.1109/ICAR53236.2021.9659413.

Ullah, Z., Al-Turjman, F., Mostarda, L., 2020. Cognition in UAV-aided 5G and beyond communications: A survey. IEEE Trans. Cogn. Commun. Netw. 6 (3), 872–891. http://dx.doi.org/10.1109/TCCN.2020.2968311.

Van Hasselt, H., Guez, A., Silver, D., 2016. Deep reinforcement learning with double q-learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 30. http://dx.doi.org/10.1609/aaai.v30i1.10295.

Zeng, Y., Wu, Q., Zhang, R., 2019. Accessing from the sky: A tutorial on UAV communications for 5G and beyond. Proc. IEEE 107 (12), 2327–2375. http://dx.doi.org/10.1109/JPROC.2019.2952892.

Zhang, Y., Aliya, B., Zhou, Y., You, I., Zhang, X., Pau, G., Collotta, M., 2018. Shortest feasible paths with partial charging for battery-powered electric vehicles in smart cities. Pervasive Mob. Comput. 50, 82–93. http://dx.doi.org/10.1016/j.pmcj.2018.08.001.

Zhang, K., Yang, Z., Başar, T., 2021. Multi-agent reinforcement learning: A selective overview of theories and algorithms. In: Handbook of Reinforcement Learning and Control. Springer, pp. 321–384. http://dx.doi.org/10.1007/978-3-030-60990-0_12.

Zhang, S., Zhang, H., Di, B., Song, L., 2019. Cellular UAV-to-X communications: Design and optimization for multi-UAV networks. IEEE Trans. Wireless Commun. 18 (2), 1346–1359. http://dx.doi.org/10.1109/TWC.2019.2892131.